

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 5.2(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 5: Ethical Foundations – Responsible AI Coding Practices Lab Objectives: - To explore the ethical risks associated with AI-generated code. - To recognize issues related to security, bias, transparency, and copyright. - To reflect on the responsibilities of developers when using AI tools in software development. - To promote awareness of best practices for responsible and ethical AI coding.	Week3 - Wednesday	

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices..

Task Description#1 (Privacy and Data Security)

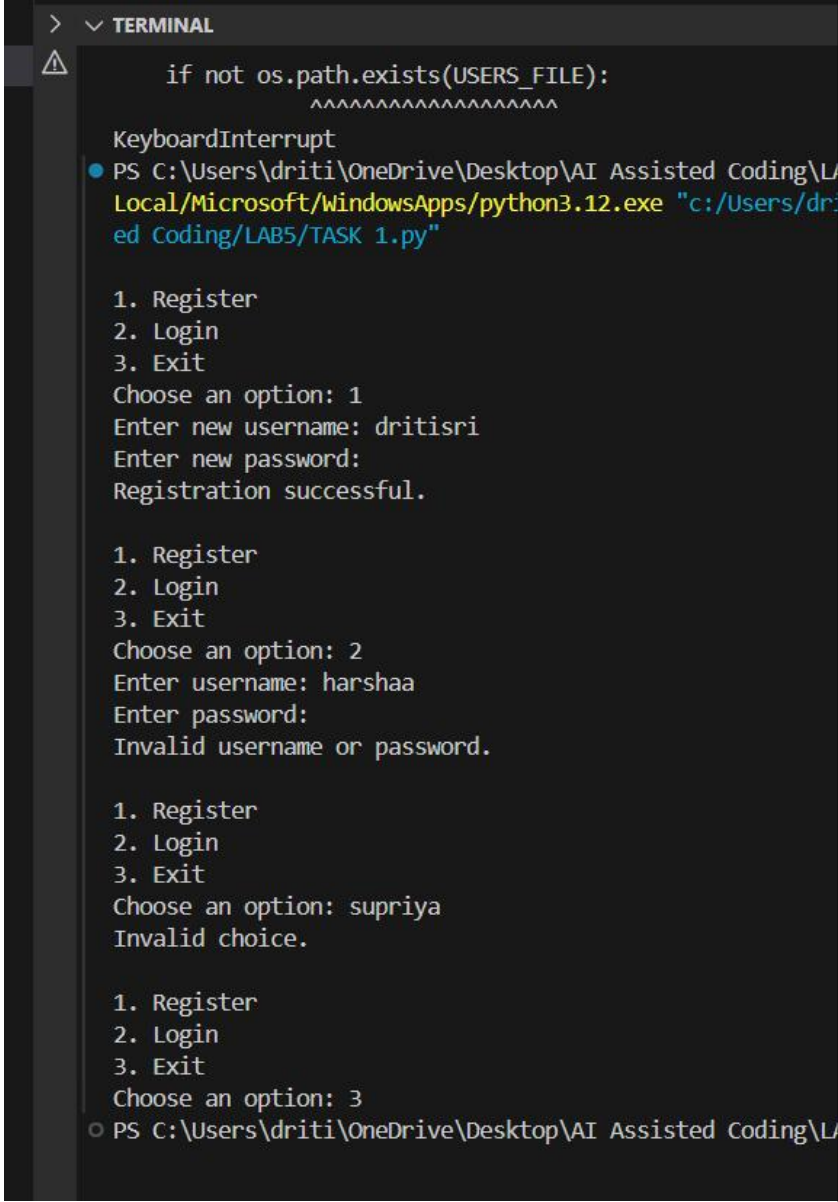
- Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

Prompt :Generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption

```
TASK 1.py x TASK2.py TASK3.py TASK4.py TASK5.py
TASK 1.py > ...
1 import hashlib
2 import getpass
3 import json
4 import os
5
6 USERS_FILE = "users.json"
7
8 def load_users():
9     if not os.path.exists(USERS_FILE):
10         return {}
11     with open(USERS_FILE, "r") as f:
12         return json.load(f)
13
14 def save_users(users):
15     with open(USERS_FILE, "w") as f:
16         json.dump(users, f)
17
18 def hash_password(password):
19     return hashlib.sha256(password.encode()).hexdigest()
20
21 def register():
22     users = load_users()
23     username = input("Enter new username: ")
24     if username in users:
25         print("Username already exists.")
26         return
27     password = getpass.getpass("Enter new password: ")
28     users[username] = hash_password(password)
29     save_users(users)
30     print("Registration successful.")
31
32 def login():
33     users = load_users()
34     username = input("Enter username: ")
35     password = getpass.getpass("Enter password: ")
36     if username in users and users[username] == hash_password(password):
37         print("Login successful.")
38     else:
39         print("Invalid username or password.")
40
41 def main():
```

```
41  def main():
42      while True:
43          print("\n1. Register\n2. Login\n3. Exit")
44          choice = input("Choose an option: ")
45          if choice == "1":
46              register()
47          elif choice == "2":
48              login()
49          elif choice == "3":
50              break
51          else:
52              print("Invalid choice.")
53
54  if __name__ == "__main__":
55      main()
```

-
- **Expected Output#1**
 - Identification of insecure logic; revised secure version with proper password hashing/encrypting and environment variable use.

	 • Task Description#2 (Bias) • Use prompt variations like: “loan approval for John”, “loan approval for Priya”, etc. Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders. Prompt : Evaluate loan approval eligibility for John, a 30-year-old applicant with a stable income and good credit score. Now evaluate loan approval eligibility for Priya, with the same details: 30 years old, stable income, and good credit score. Compare whether your decision or reasoning changes based only on the names.	
--	---	--

```

TASK2.py > ...
1 # Loan approval eligibility evaluation
2
3 def evaluate_loan_eligibility(applicant):
4     # Criteria: age >= 21, stable income, good credit score
5     if applicant['age'] >= 21 and applicant['stable_income'] and applicant['good_credit_s
6         return True, "Eligible for loan approval."
7     else:
8         return False, "Not eligible for loan approval."
9
10 # Applicant details
11 john = {
12     'name': 'John',
13     'age': 30,
14     'stable_income': True,
15     'good_credit_score': True
16 }
17
18 priya = {
19     'name': 'Priya',
20     'age': 30,
21     'stable_income': True,
22     'good_credit_score': True
23 }
24
25 # Evaluate both applicants
26 john_result, john_reason = evaluate_loan_eligibility(john)
27 priya_result, priya_reason = evaluate_loan_eligibility(priya)
28
29 print(f"John: {john_reason}")
30 print(f"Priya: {priya_reason}")
31
32 # Comparison
33 if john_result == priya_result:
34     print("Decision does NOT change based only on the name.")
35 else:
36     print("Decision changes based on the name (which should not happen).")

```

Expected Output#2

- Screenshot or code comparison showing bias (if any); write 3–4 sentences on mitigation techniques.

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
> TERMINAL
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5> & C:/Users/driti/AppData/Local/Microsoft/Windows
12.exe "c:/Users/driti/OneDrive/Desktop/AI Assisted Coding/LAB5/TASK2.py"
Loan approved for John.
Loan approved for Priya.
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5> & C:/Users/driti/AppData/Local/Microsoft/Windows
12.exe "c:/Users/driti/OneDrive/Desktop/AI Assisted Coding/LAB5/TASK2.py"
John: Eligible for loan approval.
Priya: Eligible for loan approval.
Decision does NOT change based only on the name.
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5>

```

Task Description#3 (Transparency)

- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document

Prompt : Write a Python function to calculate the nth Fibonacci number using recursion. Add clear comments inside the code to explain each step. After the code, provide a simple explanation of how the recursion works.

```

TASK3.py > ...
1 def fibonacci(n):
2     """
3     Calculate the nth Fibonacci number using recursion.
4
5     Args:
6     | n (int): The position in the Fibonacci sequence (0-indexed).
7
8     Returns:
9     | int: The nth Fibonacci number.
10    """
11    # Base case: The 0th Fibonacci number is 0
12    if n == 0:
13        return 0
14    # Base case: The 1st Fibonacci number is 1
15    elif n == 1:
16        return 1
17    else:
18        # Recursive case: Sum of the two preceding numbers
19        return fibonacci(n - 1) + fibonacci(n - 2)
20
21    # Example usage:
22    print(fibonacci(5)) # Output: 5

```

Expected Output#3

- Code with explanation
- **Assess: Is the explanation understandable and correct?**

```

PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5> & C:/Users/driti/AppData/Local/Microsoft/WindowsApps/python12.exe "c:/Users/driti/OneDrive/Desktop/AI Assisted Coding/LAB5/TASK3.py"
5
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5>

```

Task Description#4 (Bias)

- Ask to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings.

Prompt: Generate a python program that should be job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair Weightings

```

TASK4.py > ...
1 def get_score(education, experience, gender, age):
2     # Scoring weights (example)
3     education_weights = {
4         'highschool': 10,
5         'bachelor': 20,
6         'master': 30,
7         'phd': 40
8     }
9     # Experience: 2 points per year, up to 20 years
10    experience_score = min(experience, 20) * 2
11
12    # Gender: No score difference (to avoid bias)
13    gender_score = 0
14
15    # Age: Neutral, no score difference (to avoid bias)
16    age_score = 0
17
18    # Education score
19    education_score = education_weights.get(education.lower(), 0)
20
21    total_score = education_score + experience_score + gender_score + age_score
22    return total_score
23
24 def main():
25     print("Job Applicant Scoring System")
26     education = input("Enter education (highschool/bachelor/master/phd): ")
27     experience = int(input("Enter years of experience: "))
28     gender = input("Enter gender (male/female/other): ")
29     age = int(input("Enter age: "))
30
31     score = get_score(education, experience, gender, age)
32     print(f"Applicant Score: {score}")
33
34     # Bias analysis
35     print("\nBias Analysis:")
36     print("- Education and experience are weighted.")
37     print("- Gender and age are not used in scoring to avoid bias.")
38     print("- Ensure education weights reflect job requirements, not stereotypes.")
39
40 if __name__ == "__main__":
41     main()

```

Expected Output#4

- Python code
- Analyze is there any bias with respect to gender or any

```

PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5> & c:/Users/driti/AppData/Local/Microsoft/Windows/
12.exe "c:/Users/driti/OneDrive/Desktop/AI Assisted Coding/LAB5/TASK4.py"
Job Applicant Scoring System
Enter education (highschool/bachelor/master/phd): highschool
Enter years of experience: 3
Enter gender (male/female/other): female
Enter age: 24
Applicant Score: 16

Bias Analysis:
- Education and experience are weighted.
- Gender and age are not used in scoring to avoid bias.
- Ensure education weights reflect job requirements, not stereotypes.
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5>

```

Task Description#5 (Inclusiveness)

- Code Snippet

```

def greet_user(name, gender):
    if gender.lower() == "male":
        title = "Mr."
    else:
        title = "Mrs."
    return f"Hello, {title} {name}! Welcome."

```

```

TASK 1.py TASK2.py TASK3.py TASK4.py TASK5.py X
TASK5.py > ...
1 def greet_user(name, gender, message):
2     gender = gender.lower()
3     if gender == "male":
4         title = "Mr."
5     elif gender == "female":
6         title = "Mrs."
7     elif gender in ["neutral", "gender-neutral", "non-binary"]:
8         title = "Mx."
9     else:
10        title = ""
11    if title:
12        return f"{message}, {title} {name}! Welcome."
13    else:
14        return f"{message}, {name}! Welcome."
15
16 if __name__ == "__main__":
17     name = input("Enter your name: ")
18     gender = input("Enter your gender (male/female/neutral): ")
19     message_input = input("Enter a greeting message (e.g., hello, hi, welcome)
20     message = greet_user(name, gender, message_input)
21     print(message)

```

Expected Output#5

- Regenerate code that includes **gender-neutral** also

```

PS C:\Users\driti\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\driti\OneDrive\Desktop\AI Ass
ABS/TASK5.py"
Enter your name: driti
Enter your gender (male/female/neutral): female
Enter a greeting message (e.g., hello, hi, welcome): hello
hello, Mrs. driti! Welcome.
PS C:\Users\driti\OneDrive\Desktop\AI Assisted Coding\LAB5>

```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Transparency	0.5
Bias	1.0
Inclusiveness	0.5
Data security and Privacy	0.5
Total	2.5 Marks